

Universidad Nacional del Nordeste
Facultad de Ciencias Exactas y Naturales y Agrimensura



LICENCIATURA EN SISTEMAS DE INFORMACIÓN

TEORÍA DE LA COMPUTACIÓN

Analizador Sintactico — MapLang

Informe de investigación

Integrantes:

Riveros, Lautaro — D.N.I.: 45.903.006

Riveros, Maximo — D.N.I.: 45.643.192

Scetti, Santiago — D.N.I.: 43.752.065

Turtola, Sabrina — D.N.I.: 44.000.850

1. Resumen

Se presenta el diseño y la implementación de **MapLang**, un **lenguaje de dominio específico** orientado a la descripción textual y no ambigua de mapas de videojuegos del género **dungeon-crawler**.

Se define formalmente una **gramática libre de contexto** de veinte producciones que satisface la condición **LL(1)**, lo cual habilita la construcción de un **analizador léxico manual** y un **analizador sintáctico descendente recursivo**, ambos implementados en **Python**.

El sistema construye un **Árbol Sintáctico Abstracto (AST)** tipado, aplica un conjunto de **validaciones semánticas** adicionales (límites del mapa, unicidad de coordenadas, referencias válidas entre salas) y genera, a partir del árbol, una representación gráfica del mapa mediante la biblioteca **matplotlib**.

Finalmente, se diseñaron catorce **casos de prueba** que se validan mediante una suite automatizada de cincuenta y dos **pruebas unitarias y de integración**. Los resultados demuestran que el analizador reconoce correctamente el lenguaje, rechaza entradas mal formadas y permite una **recuperación básica de errores**.

2. Introducción

Los **lenguajes de dominio específico (DSL)** permiten reducir la brecha entre una necesidad de comunicación técnica y su representación procesable por una máquina, restringiendo el vocabulario y la sintaxis disponibles a un problema acotado.

En el desarrollo de videojuegos, la descripción de niveles y mapas constituye una tarea recurrente que en la mayoría de las herramientas comerciales se resuelve mediante editores gráficos propietarios, lo cual dificulta el versionado en sistemas de control de cambios, la generación procedural de contenido y la colaboración basada en texto plano entre diseñadores. Este trabajo presenta **MapLang**, un DSL textual para describir mapas de tipo **dungeon-crawler**: estructuras de habitaciones conectadas entre sí, cada una con un conjunto de atributos (tipo de sala, salidas disponibles, enemigo presente, nivel de dificultad, ítem y personaje no jugable).

El objetivo del trabajo es doble: por un lado, especificar formalmente una **gramática no ambigua** para este lenguaje y, por otro, construir un **analizador léxico-sintáctico** completo — con manejo de errores, generación de **AST** y una salida visual — que permita validar cadenas de entrada y demostrar de forma tangible los conceptos de la **Teoría de la Computación** abordados en la asignatura: gramáticas formales, autómatas, derivaciones, ambigüedad y técnicas de parsing.

La motivación de elegir este dominio responde a varios criterios. En primer lugar, su **riqueza gramatical**: el lenguaje requiere **estructuras anidadas** (un mapa contiene salas, cada sala contiene atributos), listas (las salidas de una sala) y **recursividad**, lo cual permite ejercitar una gramática no trivial sin recurrir a artificios. En segundo lugar, su potencial visual: a diferencia de un DSL puramente textual, **MapLang** permite renderizar el **AST** resultante como una imagen del mapa, lo que aporta una demostración de alto impacto durante la exposición oral.

Finalmente, existe aplicabilidad real: editores como Tiled Map Editor o el sistema de tilemaps de Godot resuelven problemas análogos en la industria. El resto del informe se organiza como sigue. La Sección 3 repasa los conceptos teóricos de gramáticas formales y autómatas empleados como marco de referencia. Además, presenta el diseño formal de la gramática de **MapLang**. La Sección 5 describe la arquitectura de la implementación y expone los casos de prueba y resultados obtenidos. La Sección 7 discute las dificultades encontradas y posibles extensiones y concluye el trabajo.

3. Marco Teórico

El **analizador léxico** de **MapLang** puede modelarse conceptualmente como un **autómata de estados finitos determinístico** $M = (Q, \Sigma, \delta, q_0, F)$ que recorre el **código fuente** carácter a carácter, donde cada “**categoría de token**” corresponde a una clase de caminos de aceptación distinta dentro del diagrama de estados implícito en la **función tokenize**.

Si bien la implementación no instancia una **tabla de transición explícita** — por simplicidad y legibilidad del código se opta por una implementación manual basada en inspección de caracteres —, el comportamiento es funcionalmente equivalente al de un **AEF determinístico**: cada decisión depende únicamente del carácter actual y

de un estado interno acotado (por ejemplo, “dentro de una cadena”, “leyendo un identificador”).

3.1 Gramáticas formales

Siguiendo la formalización clásica, una **gramática formal** se define como una **cuaterna** $G = (N, T, P, S)$, donde N es un conjunto finito de **símbolos no terminales**, T un conjunto finito de **símbolos terminales** con $N \cap T = \emptyset$, P un conjunto finito de **producciones** de la forma $\alpha \rightarrow \beta$, y $S \in N \cup T$ el símbolo distinguido.

El lenguaje generado por G se define como $L(G) = \{ \omega \in T^* \mid S \Rightarrow^* \omega \}$. La **Jerarquía de Chomsky** clasifica a las gramáticas en cuatro tipos según la forma de sus producciones, desde las **gramáticas sin restricción** (Tipo 0) hasta las **gramáticas regulares** (Tipo 3).

El lenguaje MapLang se especifica mediante una **Gramática Libre de Contexto (GLC, Tipo 2)**: todas sus producciones tienen la forma $A \rightarrow \omega$ con $A \in N$ y $\omega \in (N \cup T)^*$, sin restricción de contexto sobre los símbolos adyacentes.

3.2 Derivaciones, árboles y ambigüedad

Una **derivación** es la secuencia de **formas sentenciales** obtenida al reescribir sucesivamente un no terminal mediante las producciones de G , comenzando en S . En particular, una **derivación por izquierda** reescribe en cada paso el símbolo no terminal más a la izquierda de la forma sentencial actual.

Una **gramática libre de contexto** es **ambigua** si y solo si existe al menos una sentencia de $L(G)$ con dos o más **derivaciones por izquierda** distintas. Esto se traduce en dos **árboles de derivación** estructuralmente diferentes (o estructuralmente iguales pero con distintos rótulos en sus nodos internos).

3.3 Autómatas de estados finitos y su rol en el lexer

El **analizador léxico** de **MapLang** puede modelarse conceptualmente como un **autómata de estados finitos determinístico (AEF)** $M = (Q, \Sigma, \delta, q_0, F)$, que recorre el **código fuente** carácter a carácter. Cada **categoría de token** corresponde a una clase de caminos de aceptación distinta dentro del diagrama de estados implícito en la función **tokenize**.

Si bien la implementación no instancia una **tabla de transición explícita** —por simplicidad y legibilidad del código se opta por una implementación manual basada en inspección de caracteres—, el comportamiento es funcionalmente equivalente al de un **AEF determinístico**. Cada decisión depende únicamente del carácter actual y de un **estado interno acotado** (por ejemplo, “dentro de una cadena” o “leyendo un identificador”).

3.4 Técnicas de parsing

Para el **reconocimiento sintáctico** se evaluaron tres estrategias: **parser descendente recursivo**, **parser LL(1) con tabla**, y **parsers ascendentes (SLR/LALR)**.

Se seleccionó el **parser descendente recursivo**, dado que la **gramática de MapLang** es **LL(1)**. Esta técnica ofrece la ventaja de una correspondencia directa y legible entre cada no-terminal de la gramática y una función del programa, facilitando tanto la implementación manual como el **control granular** sobre el **reporte de errores**, lo cual constituye un requisito explícito del trabajo.

4. Diseño de la Gramática

4.1 Clase gramatical

La gramática definida para **MapLang** es una **Gramática Libre de Contexto**. Tras el análisis de los conjuntos **FIRST** y **FOLLOW**, se verificó que satisface la condición **LL(1)**, lo que permite la implementación de un **parser descendente recursivo** sin necesidad de **backtracking**.

4.2 Notación BNF

La siguiente tabla presenta las veinte producciones que definen la gramática completa de MapLang.

#	No Terminal	Producción
P1	<programa>	<programa> ::= <mapa>+

P2	<mapa>	<mapa> ::= "MAP" <cadena> "SIZE" <dimension> "{" <cuerpo> "}"
P3	<cuerpo>	<cuerpo> ::= <declaracion>+
P4	<declaracion>	<declaracion> ::= <sala> <conexion>
P5	<sala>	<sala> ::= "ROOM" "(" <entero_pos> "," <entero_pos> ")" <atributo>+
P6	<atributo>	<atributo> ::= <tipo_sala> <salidas> <enemigo> <nivel> <item> <npc>
P7	<tipo_sala>	<tipo_sala> ::= "TYPE" "=" <tipo_valor>
P8	<tipo_valor>	<tipo_valor> ::= "entrada" "pasillo" "combate" "tesoro" "jefe" "tienda" "trampa"
P9	<salidas>	<salidas> ::= "EXITS" "=" "[" <lista_dir> "]"
P10	<lista_dir>	<lista_dir> ::= <direccion> ("," <direccion>)*
P11	<direccion>	<direccion> ::= "norte" "sur" "este" "oeste" "arriba" "abajo"
P12	<enemigo>	<enemigo> ::= "ENEMY" "=" <identificador>
P13	<nivel>	<nivel> ::= "LEVEL" "=" <entero_pos>
P14	<item>	<item> ::= "ITEM" "=" <identificador>
P15	<npc>	<npc> ::= "NPC" "=" <identificador>
P16	<conexion>	<conexion> ::= "CONNECT" "(" <entero_pos> "," <entero_pos> ")" ">" "(" <entero_pos> "," <entero_pos> ")" ["via" <cadena>]
P17	<dimension>	<dimension> ::= <entero_pos> "x" <entero_pos>
P18	<cadena>	<cadena> ::= "'" <caracter>* "'"
P19	<identificador>	<identificador> ::= [a-zA-Z_][a-zA-Z0-9_]*
P20	<entero_pos>	<entero_pos> ::= [0-9]+

4.3 Justificación de las decisiones de diseño

Palabras clave distintivas por atributo: La producción P6 admite seis alternativas (TYPE, EXITS, ENEMY, LEVEL, ITEM, NPC), cada una identificada por una **palabra reservada** única. Esta decisión elimina cualquier conflicto de predicción: el conjunto **FIRST** de cada alternativa es unitario y disjunto del resto, por

lo que el **parser** puede elegir la producción correcta observando un único **token** de **lookahead**.

Orden libre de atributos: Inicialmente se consideró una gramática que fijara el orden de aparición de los atributos dentro de una **ROOM**. Se descartó esta alternativa porque incrementaba artificialmente el número de **producciones** sin aportar expresividad y porque impondría una restricción arbitraria. La **producción P6**, al permitir que **<atributo>** se repita en cualquier orden mediante la **clausura positiva** de **P5**, resuelve este problema sin introducir **ambigüedad**.

Conexión con etiqueta opcional: La cláusula [**"via"** **<cadena>**] de **P16** introduce la única parte opcional de toda la gramática. Se decidió expresarla como una elección entre presencia y ausencia, simplificando la función **parse_conexion** a una única verificación condicional. Esta decisión es **LL(1)** segura, dado que el **token** **"via"** no pertenece a **FOLLOW(<conexion>)**.

Enteros no negativos: Se introdujo explícitamente **<entero_pos>** en lugar de un **<entero>** genérico, reforzando que las coordenadas y niveles de dificultad deben ser valores no negativos. Aunque esto no cambia el conjunto de cadenas reconocidas por el **lexer**, clarifica la intención semántica de la **producción** en la documentación de la gramática.

4.4 Recursividad y anidamiento

La **recursividad** de la gramática se manifiesta en tres **producciones**: **P1** (**<programa> ::= <mapa>+**, permite un número arbitrario de mapas en un mismo programa), **P3** (**<cuerpo> ::= <declaracion>+**, un número arbitrario de salas y conexiones por mapa) y **P10** (**<lista_dir> ::= <direccion> ("," <direccion>)***, listas de longitud arbitraria de direcciones de salida).

El **anidamiento** se observa en que un **<mapa>** contiene un **<cuerpo>** que contiene **<sala>**, la cual a su vez contiene múltiples **<atributo>**; y en que un **<atributo>** de tipo **<salidas>** contiene una **<lista_dir>**.

4.5 Justificación de no ambigüedad

De acuerdo a la definición de **ambigüedad** expuesta, una **GLC** es ambigua si y sólo si alguna sentencia de su lenguaje admite dos **derivaciones por izquierda**

distintas. Se argumenta la **no ambigüedad** de **MapLang** por inspección de cada **producción** con más de una alternativa:

- **P4** (<sala> | <conexion>): las alternativas comienzan, respectivamente, con las palabras clave ROOM y CONNECT, mutuamente excluyentes.
- **P6** (las seis variantes de <atributo>): cada una comienza con una palabra clave distinta.
- **P8 y P11** (<tipo_valor> y <direccion>): cada alternativa es un **terminal literal** distinto; no existe **superposición léxica** entre ellos porque el **lexer** asigna a cada palabra reservada una única categoría de **token**.

Dado que en ningún punto de una **derivación** existe más de una producción aplicable para un mismo **prefijo de entrada**, se concluye que toda sentencia de **L(G)** posee una única derivación por izquierda y, por lo tanto, un único **árbol de derivación**: la gramática no es ambigua. Esta propiedad se verifica empíricamente en la prueba automatizada **test_same_input_always_same_ast_structure**.

4.6 Verificación de la condición LL(1)

Una gramática es **LL(1)** si para cada no terminal con más de una producción, los conjuntos **FIRST** de cada alternativa son disjuntos entre sí y, en caso de que alguna alternativa derive λ , dicho conjunto es también disjunto de **FOLLOW** del no terminal.

Para **MapLang**, la verificación de esta propiedad se resume en:

- **FIRST(<sala>) = {ROOM}, FIRST(<conexion>) = {CONNECT}**: ambos disjuntos.
- **FIRST(<tipo_sala>) = {TYPE}, FIRST(<salidas>) = {EXITS}, FIRST(<enemigo>) = {ENEMY}, FIRST(<nivel>) = {LEVEL}, FIRST(<item>) = {ITEM}, FIRST(<npc>) = {NPC}**: los seis conjuntos son disjuntos entre sí.

Para la parte opcional de la producción **P16**, se verifica que **FOLLOW(<conexion>) = FIRST(<declaracion>) $\cup$ {""} = {ROOM, CONNECT, ""}**. Dado que el token "via" no pertenece a **FOLLOW(<conexion>)**, la decisión de tomar o no la cláusula opcional se resuelve con un único token de **lookahead** sin ambigüedad.

Por último, aunque no se presenta el cálculo completo de **FIRST/FOLLOW** para las veinte producciones por razones de espacio, el razonamiento anterior cubre los

únicos puntos de decisión no triviales de la gramática. El resto de las producciones no requiere elección entre alternativas (son únicas) o se resuelve trivialmente por la palabra clave inicial.

4.7 Ejemplo de cadena válida e inválida

Cadena válida: la «Mazmorra del Dragón».

```
MAP "Mazmorra_del_Dragon" SIZE 4x1 {  
  ROOM(0,0) TYPE=entrada EXITS=[este]  
  ROOM(1,0) TYPE=combate ENEMY=goblin LEVEL=2 EXITS=[este, oeste]  
  ROOM(2,0) TYPE=tesoro ITEM=pocion_vida EXITS=[este, oeste]  
  ROOM(3,0) TYPE=jefe ENEMY=dragon LEVEL=10 EXITS=[oeste]  
  CONNECT (0,0)->(1,0) via "corredor_este"  
  CONNECT (1,0)->(2,0) via "puerta_madera"  
  CONNECT (2,0)->(3,0) via "puerta_secreta"  
}
```

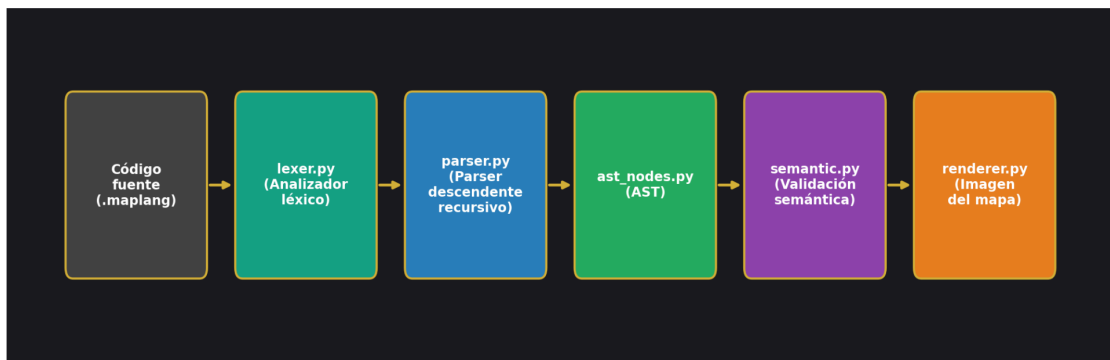
Cadena inválida: faltan paréntesis y coordenadas.

```
MAP "Mapa_Roto" SIZE 3x3 {  
  ROOM(0,0 TYPE=entrada  
  ROOM(1,) TYPE=combate  
  CONNECT (0,0)-> via "x"  
}
```

5. Implementación

La implementación se realizó en **Python 3.12**, por su **legibilidad**, su amplio ecosistema de bibliotecas y su facilidad para integrar la visualización del **AST** y del **mapa generado**.

El sistema se organiza en cinco **módulos independientes** (Fig. 1): **lexer.py**, **parser.py**, **ast_nodes.py**, **semantic.py** y **renderrer.py**, integrados por **main.py**.



5.1 Analizador léxico

El **lexer (lexer.py)** se implementó de forma **manual** —sin bibliotecas como **Flex**, **PLY** o **ANTLR**—, recorriendo el **código fuente** carácter a carácter.

Cada categoría de **token** se reconoce mediante **inspección directa de prefijos**: cadenas delimitadas por comillas dobles, secuencias de dígitos (interpretadas como literales de dimensión **SIZE**), e identificadores clasificados retroactivamente como **palabra clave**, **valor de tipo de sala**, **dirección**, o **identificador genérico** según pertenezcan o no a los conjuntos **KEYWORDS**, **TYPE_VALS** o **DIRECTIONS**. Los **espacios en blanco** y los **comentarios de línea** (`// ...`) se descartan sin generar token.

La siguiente tabla resume las categorías de token reconocidas. Cada token registra **línea** y **columna** de origen, información reutilizada en el **reporte de errores** tanto léxicos como sintácticos.

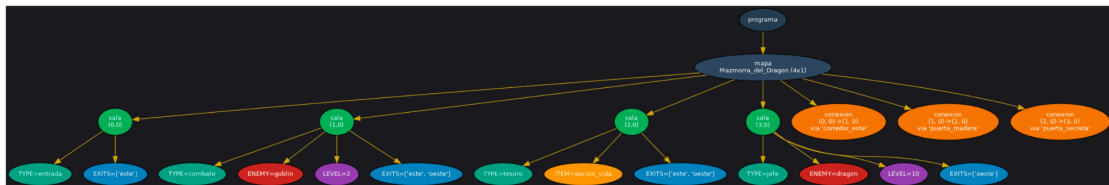
Categoría	Ejemplos
KEYWORD	MAP, ROOM, CONNECT, TYPE, EXITS, ENEMY, LEVEL, ITEM, NPC, SIZE, via
TYPE_VAL	entrada, pasillo, combate, tesoro, jefe, tienda, trampa
DIRECTION	norte, sur, este, oeste, arriba, abajo
STRING	"Mazmorra_1", "puerta_madera"
SIZE	8x8, 10x12, 5x5
INTEGER	0, 1, 3, 10, 255

IDENTIFIER goblin, espada_magica, mercader

PUNCT { } () [] = , ->

Ante un carácter **no reconocido** o una cadena sin cerrar, el lexer lanza una **excepción** `LexError` que incluye número de línea y columna.

5.2 Analizador sintáctico



El **parser** (`parser.py`) implementa un **descendente recursivo**: cada **no-terminal** de la gramática (Tabla 1) se traduce en un método `parse_X` de la clase **Parser**, que consume **tokens** de una lista y construye el nodo de **AST** correspondiente.

El siguiente listado muestra la implementación de `parse_conexion`, que ilustra la resolución de la parte opcional mediante un único **lookahead**.

Fragmento de `parse_conexion`.

```
302 def parse_conexion(self) -> Conexion:
303     """<conexion> ::= "CONNECT" "(" <entero_pos> "," <entero_pos> ")" "->"
304     "(" <entero_pos> "," <entero_pos> ")" [ "via" <cadena> ] """
305     connect_tok = self.expect_keyword("CONNECT")
306     self.expect_punct("(")
307     x1 = self.expect_type("INTEGER", "un entero (coordenada x de origen)")
308     self.expect_punct(",")
309     y1 = self.expect_type("INTEGER", "un entero (coordenada y de origen)")
310     self.expect_punct("(")
311     self.expect_punct("->")
312     self.expect_punct("(")
313     x2 = self.expect_type("INTEGER", "un entero (coordenada x de destino)")
314     self.expect_punct(",")
315     y2 = self.expect_type("INTEGER", "un entero (coordenada y de destino)")
316     self.expect_punct("(")
317
318     etiqueta = None
319     # Decisión LL(1): "via" no pertenece a FOLLOW(<conexion>), por lo que
320     # un único token de lookahead basta para decidir si la parte opcional
321     # está presente.
322     if self.check_keyword("via"):
323         self.advance()
324         etiqueta_tok = self.expect_type("STRING", "una cadena (etiqueta de la conexión)")
325         etiqueta = etiqueta_tok.value
326
327     return Conexion(
328         origen=(int(x1.value), int(y1.value)),
329         destino=(int(x2.value), int(y2.value)),
330         etiqueta=etiqueta,
331         line=connect_tok.line,
332     )
333
```

El **manejo y recuperación de errores** se gestiona de la siguiente manera: ante un token inesperado, el sistema lanza una excepción **ParseError** que detalla la línea, la columna y un mensaje orientado al usuario (p. ej., "se esperaba ')'" pero se encontró 'TYPE' (KEYWORD)").

Cuando el parser se invoca con la configuración **collect_errors=True**, los errores producidos dentro de una **<sala>** o **<conexion>** no abortan el análisis. En su lugar, el parser registra el error y ejecuta una **rutina de sincronización en modo pánico**, descartando tokens hasta encontrar el inicio de la siguiente declaración (**ROOM/CONNECT**) o el cierre de bloque ("}"). Esto permite informar múltiples errores sintácticos en una sola ejecución, evitando detenerse tras el primer fallo.

5.3 Árbol Sintáctico Abstracto

El **AST** se modela en **ast_nodes.py** mediante **dataclasses** de **Python** (Programa, Mapa, Sala, Conexion y los seis tipos de atributo), lo que provee automáticamente **comparación estructural** —usada extensivamente en los tests— y un método **to_dict()** para la **serialización a JSON**.

El sistema admite tres **representaciones del AST**, todas disponibles desde **main.py**: **árbol textual** indentado en consola, **exportación a JSON**, y un **diagrama Graphviz**.

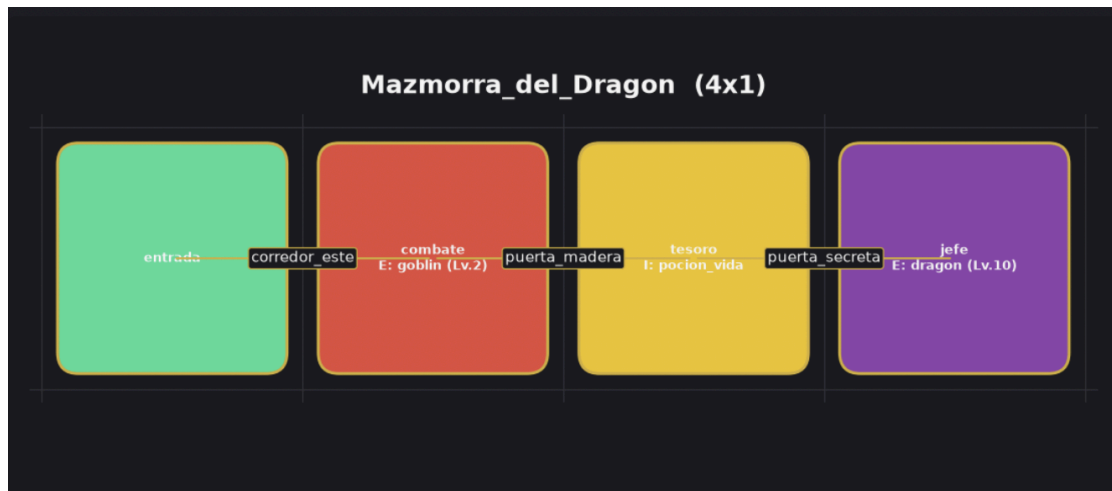
5.4 Validación semántica

Más allá de lo que una **GLC** puede expresar convenientemente, se implementó en **semantic.py** un recorrido del **AST** que verifica cuatro **reglas adicionales de buena formación**.

Estas reglas son: (S1) las coordenadas de cada sala deben estar dentro de los **límites** declarados por **SIZE**; (S2) no puede haber dos salas con la misma coordenada; (S3) toda **CONNECT** debe referenciar salas existentes; y (S4) un atributo no puede repetirse dentro de la misma sala.

Estas reglas dependen del **contexto global** del mapa —no de una única producción— por lo que se verifican como una **etapa posterior al parseo**, análoga al **análisis semántico** de un compilador convencional.

5.5 Renderizador

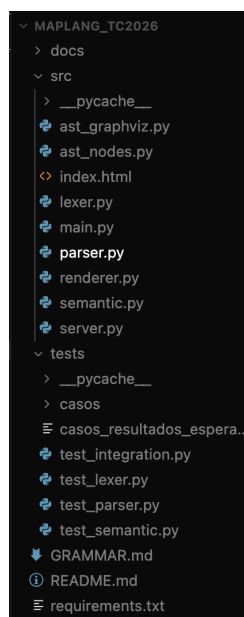


El módulo **renderer.py** recorre el **AST** de un Mapa y construye, con **matplotlib**, una grilla donde cada sala se dibuja como una celda coloreada según su **TYPE**, anotada con sus atributos.

Asimismo, representa cada **CONNECT** como una línea entre los centros de las salas correspondientes. Esta es la **salida visual** central del sistema y el punto de mayor impacto durante la demostración en vivo.

5.6 Estructura modular y control de versiones

Estructura de archivos del proyecto.



6. Casos de Prueba y Resultados

6.1 Diseño de los casos

Se diseñaron **catorce casos de prueba** almacenados como archivos `.maplang` independientes en `tests/casos/`. Se incluyen **ocho casos válidos** que contemplan escenarios como dimensiones límite de 1x1, múltiples mapas en un mismo programa y la verificación del orden libre de atributos.

Adicionalmente, se desarrollaron **seis casos inválidos** que cubren las tres **etapas de validación**: **errores léxicos** (carácter no reconocido, cadena sin cerrar), **errores sintácticos** (paréntesis faltante, coordenada faltante, conexión incompleta, cuerpo vacío) y **errores semánticos** (coordenada duplicada, sala fuera de límites, conexión a sala inexistente).

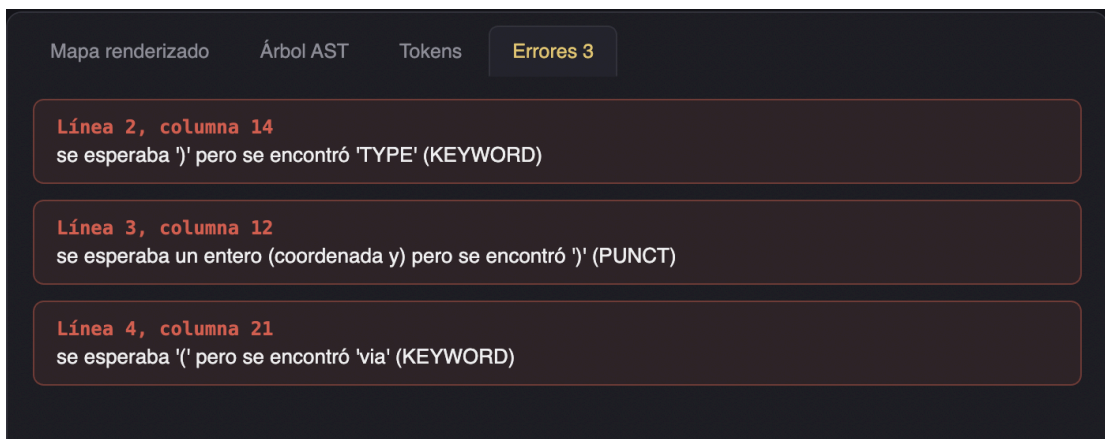
La siguiente tabla resume un subconjunto representativo de estas pruebas.

Caso	Validación	Result
<code>valido_01_lineal</code>	– (válido)	Aceptada
<code>valido_06_limite_minimo</code>	– (límite 1x1)	Aceptada
<code>valido_08_orden_libre</code>	– (válido)	Rechazada
<code>invalido_01_caracter</code>	Léxica	Rechazada
<code>invalido_02_cadena_sin_cerrar</code>	Léxica	Rechazada
<code>invalido_03_sintacticos_multiples</code>	Sintáctica (×3)	Rechazada
<code>invalido_05_sem_multiples</code>	Semántica (×3)	Rechazada
<code>invalido_06_cuerpo_vacio</code>	Sintáctica	Rechazada

6.2 Ejemplo de salida ante entrada inválida

Para el caso `invalido_05_errores_semanticos.maplang` (una **sala duplicada** en (0,0), una sala en (5,5) **fuera de los límites** de un mapa 2×2, y una **conexión hacia la coordenada inexistente** (9,9)), el sistema reporta:

Salida real del sistema ante errores semánticos.



Los **tres errores** se reportan en **una sola ejecución**, cada uno con su número de línea correspondiente.

6.3 Suite automatizada de pruebas

Se implementó una **suite de 52 pruebas automatizadas** utilizando el módulo **unittest de Python**, organizada en **cuatro archivos**.

Las **pruebas de integración** ejecutan el **pipeline completo** (**lexer** → **parser** → **semántica** → **render**) sobre los catorce casos de tests/casos/. Estas verifican que los **casos válidos** se acepten sin errores y que los **inválidos** produzcan al menos un error en la etapa correspondiente, finalizando todas con **resultado OK**.

Archivo	Cobertura	N
<code>test_lexer.py</code>	Tokens, errores léxicos	20
<code>test_parser.py</code>	Producciones válidas/inválidas, recuperación, no-ambigüedad	18
<code>test_semantic.py</code>	Reglas semánticas S1-S4	10
<code>test_integration.py</code>	Pipeline end-to-end, render	4
TOTAL		52

7. Discusión y Análisis

Dificultades encontradas: El principal **desafío de diseño** fue evitar la **ambigüedad** en la producción de <atributo>: una primera aproximación que no fijara **palabras clave distintivas** por atributo generaría **conflictos de predicción** al permitir que dos alternativas compartieran el mismo prefijo. Esto se resolvió exigiendo que cada atributo comience con una **palabra reservada exclusiva**.

Un segundo desafío surgió al decidir la **estrategia de recuperación de errores**: una implementación naïve que abortara en el primer error sintáctico resultaba poco útil para un usuario que comete varios errores de tipeo en un mismo archivo; la **sincronización en modo pánico** hacia el inicio de la siguiente declaración resultó ser la solución más simple que preserva la mayor cantidad de información útil.

Decisiones de diseño relevantes: Se optó deliberadamente por una **implementación manual** del **lexer** y el **parser** —sin herramientas como ANTLR o PLY— en línea con el requisito de la cátedra de comprensión y defensa conceptual de la implementación. Esto exigió mayor cuidado en el manejo de casos borde (por ejemplo, distinguir un literal **SIZE** del tipo 8x8 de un entero seguido de un identificador que casualmente comienza con x), pero permitió un control total sobre los mensajes de error.

Posibles extensiones: El lenguaje actual excluye deliberadamente la lógica de eventos o triggers dentro de las habitaciones, la **herencia entre tipos de sala**, y los mapas **multi-piso**. Una extensión natural consistiría en agregar un nuevo no-terminal <evento> anidado dentro de <sala>, lo cual no rompería la condición **LL(1)** si se reutiliza el mismo patrón de palabras clave distintivas.

Asimismo, podría incorporarse un **análisis de alcanzabilidad** sobre el grafo de conexiones (verificando que todas las salas sean alcanzables desde la sala de tipo entrada) como una quinta regla semántica.

8. Conclusiones

Se diseñó e implementó un **analizador léxico-sintáctico** completo para **MapLang**, un **lenguaje de dominio específico** para la descripción no ambigua de mapas de video-juegos. La gramática propuesta, de veinte producciones, se verificó formalmente como **libre de contexto**, no ambigua y **LL(1)**, lo que permitió la construcción de un **parser descendente recursivo** manual con manejo y recuperación de errores.

El sistema completo — **lexer**, **parser**, **AST**, **validación semántica** y **renderizador** — se valida mediante una suite de 52 pruebas automatizadas y 14 casos de prueba documentados, todos con resultado correcto.

El trabajo permitió aplicar de manera integrada los conceptos centrales de la asignatura: **gramáticas formales**, **derivaciones**, **ambigüedad**, **autómatas** y **técnicas de parsing**, además de competencias profesionales de trabajo colaborativo y documentación técnica. Como aprendizaje de equipo, se destaca la importancia de fijar decisiones de diseño gramatical (como las **palabras clave distintivas**) en etapas tempranas del proyecto, dado que revertirlas una vez avanzada la implementación del parser resulta costoso.

9. Bibliografía

[1] Cátedra de Teoría de la Computación, Lingüística Matemática y Gramáticas Formales (material de cátedra), Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste, 2026.

[2] Cátedra de Teoría de la Computación, Máquinas o Autómatas de Estados Finitos (material de cátedra), Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste, 2026.

[3] Cátedra de Teoría de la Computación, Autómatas de Tape o de Cinta (material de cátedra), Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste, 2026.

[4] Cátedra de Teoría de la Computación, Teoría Turing (material de cátedra), Facultad de Ciencias Exactas y Naturales y Agrimensura, Universidad Nacional del Nordeste, 2026.